returns `Behavior.same` and logs that the message was not handled. This is done with `.receiveMessagePartial`:

```
Class MonitoringExample ... {

  "Among two actors, NO parent/child related, the watcher" must {
    "be able to be notified with Terminated when watched actor stops" in {
      val watcher = spawn(SimplifiedFileWatcher())
      val logprocessor = spawn(Behaviors.receiveMessagePartial[String] {
        case "stop" =>
          Behaviors.stopped
      })

      watcher.ref ! SimplifiedFileWatcher.Watch(logprocessor.ref)    ❶

      LoggingTestKit.info("terminated").expect {                     ❷
        logprocessor ! "stop"                                        ❸
      }
    }

  ...
}
```

❶ Watches the log processor actor

❷ Proves the message “terminated” is logged by the watcher

❸ Sends “stop” to the actor

Here, the simplified actor representing the log processor is watched by including it in the `Watch` message when it is sent to the watcher. Then a “stop” message is sent to the log processor, which triggers the `Terminated` signal to the watcher and thus the logging of “terminated”.

The last important thing you need to know is that if an actor is watching another actor but has not implemented a `receiveSignal`, it produces a `DeathPactException` if the watched actor terminates or fails. There are two possibilities: either you forgot to take care of this situation or you did it on purpose. If you